

Fondamentaux du Deep Learning

et Vision par Ordinateur avec PyTorch

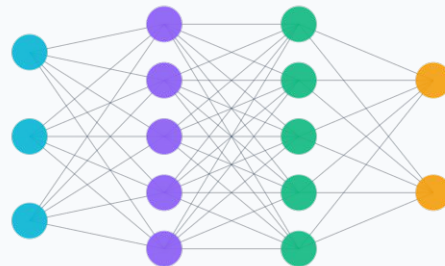
Du neurone artificiel au Transfer Learning



Qu'est-ce que le Deep Learning ?

Définition

Le Deep Learning est un sous-domaine du Machine Learning qui utilise des réseaux de neurones artificiels profonds (avec plusieurs couches) pour apprendre automatiquement des représentations complexes à partir de données brutes.



Analogie

Comme un enfant apprend à reconnaître un chat après en avoir vu des centaines, un réseau profond apprend à reconnaître des motifs en s'entraînant sur des millions d'exemples.



Vision

Reconnaissance
d'objets,
détection faciale



NLP

Traduction,
chatbots,
analyse de
sentiments



Autonomie

Conduite autonome,
robotique

Un peu d'histoire du Deep Learning



Pourquoi PyTorch ?



Open Source

Développé par Meta AI avec une communauté massive de chercheurs et développeurs. Plus de 75k stars sur GitHub.



Graphe Dynamique

Contrairement à TensorFlow v1, PyTorch permet de modifier le réseau en temps réel, facilitant le débogage.



Tensors & GPU

Manipulation de tableaux multidimensionnels avec accélération GPU native via torch.cuda. Syntaxe proche de NumPy.



Écosystème Riche

torchvision, torchaudio, torchtext, Hugging Face... Un écosystème complet pour tous les domaines du DL.

TP 1

Configuration de l'Environnement

Objectifs

- Installer PyTorch et les dépendances
- Vérifier l'accès GPU pour l'accélération
- Manipuler des tensors (création, opérations)

```
# Installation
pip install torch torchvision

# Vérification GPU
import torch
print(torch.cuda.is_available())

# Exercice : Créer un tensor 3x3
t = torch.randn(3, 3)
print(t + t)

# Addition
```

Anatomie d'un Neurone Artificiel

Un neurone artificiel imite le fonctionnement simplifié d'un neurone biologique :

1

Agrégation

Chaque entrée x_i est multipliée par un poids w_i , puis on additionne le tout avec un biais b :

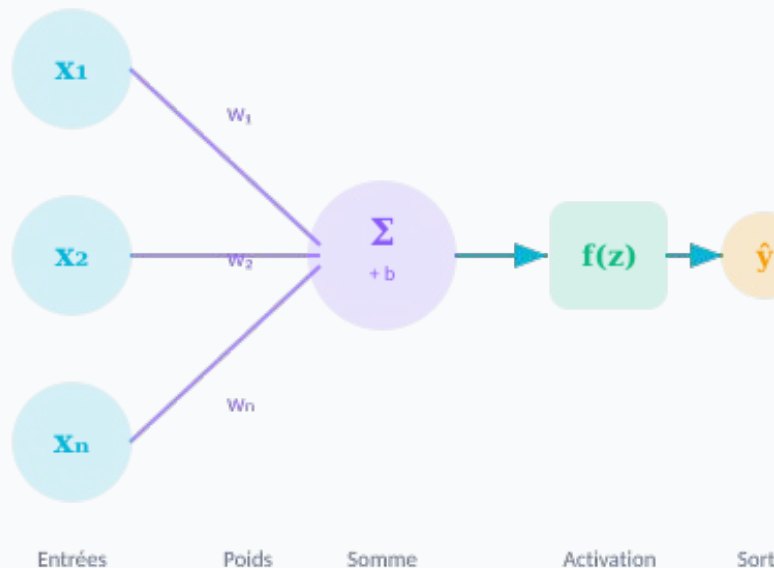
$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

2

Activation

La somme z passe dans une fonction non-linéaire $f(z)$ qui décide si le neurone s'active.

Sortie : $\hat{y} = f(z)$

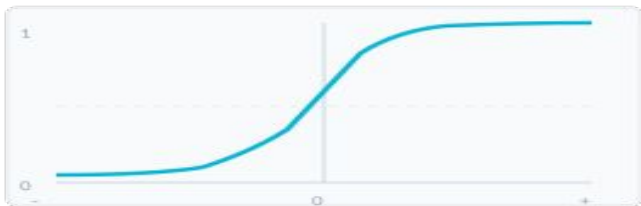


Fonctions d'Activation (1/2)

Sans activation, un réseau profond ne serait qu'une combinaison linéaire, incapable d'apprendre des relations complexes.

Sigmoid

$$\sigma(z) = 1 / (1 + e^{-z})$$



- Sortie entre 0 et 1 → probabilités
- Problème : vanishing gradient aux extrêmes

ReLU (Rectified Linear Unit)

$$f(z) = \max(0, z)$$



- Standard actuel pour les couches cachées
- Calcul rapide, pas de saturation du gradient

Fonctions d'Activation (2/2)

Tanh

$$\tanh(z) = (e^z - e^{-z}) / (e^z + e^{-z})$$

Sortie entre -1 et 1, centrée sur zéro. Meilleure que Sigmoid pour les couches cachées car les gradients sont plus forts près de 0.

Softmax

$$\sigma(z_i) = e^{z_i} / \sum_j e^{z_j}$$

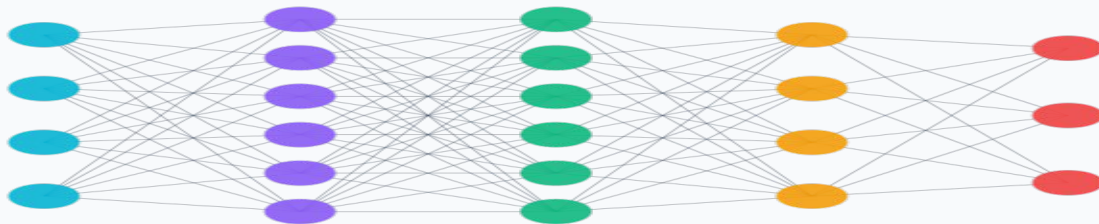
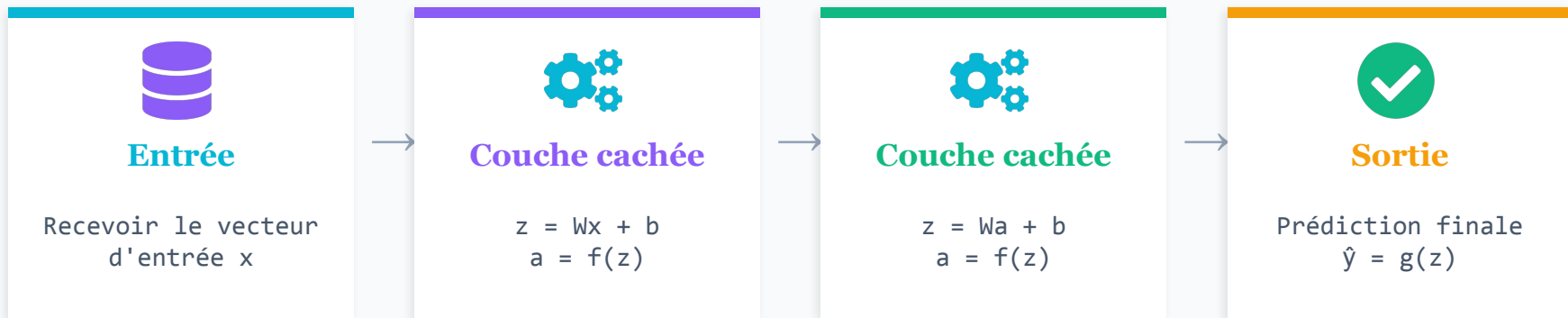
Transforme un vecteur de scores en distribution de probabilité (somme = 1). Indispensable pour la classification multi-classes en sortie.

Quelle fonction pour quel usage ?

Fonction	Formule	Usage Typique	Sortie
ReLU	$\max(0, z)$	Couches cachées (default)	$[0, +\infty)$
Sigmoid	$1/(1+e^{-z})$	Classification binaire (sortie)	$[0, 1]$
Tanh	$(e^z - e^{-z}) / (e^z + e^{-z})$	Couches cachées (alternative)	$[-1, 1]$
Softmax	$e^{z_i} / \sum_j e^{z_j}$	Classification multi-classes (sortie)	$[0,1] \sum=1$

Forward Propagation (Propagation Avant)

C'est le trajet des données de l'entrée vers la sortie. À chaque couche, on calcule une transformation : multiplication par les poids, addition du biais, puis passage dans la fonction d'activation.



TP 2

Création d'un Perceptron Simple

Objectifs

- Coder un neurone avec NumPy (sans PyTorch)
- Implémenter somme pondérée + sigmoid
- Classifier le OR logique

```
import numpy as np

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def forward(X, W, b):
    z = np.dot(X, W) + b
    return sigmoid(z)

# Dataset OR
X = np.array([[0,0],[0,1],[1,0],[1,1]])
y = np.array([0, 1, 1, 1])
w = à compléter
b=  à compléter
```



La Fonction de Coût (Loss Function)

La loss mesure à quel point le modèle se trompe. L'objectif de l'entraînement est de la minimiser. C'est le « bulletin de notes » du modèle à chaque itération.

MSE (Mean Squared Error)

$$\text{MSE} = (1/n) \sum (y_i - \hat{y}_i)^2$$

Mesure la moyenne des carrés des erreurs. Pénalise fortement les grosses erreurs (effet quadratique). Utilisée pour la régression (prédire des valeurs continues comme des prix, températures...).

Exemple : Prédiction de prix immobiliers, prévision météo, estimation d'âge

Cross-Entropy

$$\text{CE} = -\sum y_i \log(\hat{y}_i)$$

Mesure la divergence entre la distribution prédite et la distribution réelle. Pénalise les prédictions confiantes mais fausses. Utilisée pour la classification (catégories discrètes).

Exemple : Chat vs chien, spam detection, reconnaissance de chiffres manuscrits

Descente de Gradient

Imaginez-vous les yeux bandés sur une colline. Pour descendre, vous tâtez le sol autour de vous et avancez dans la direction la plus pentue. C'est exactement ce que fait la descente de gradient !

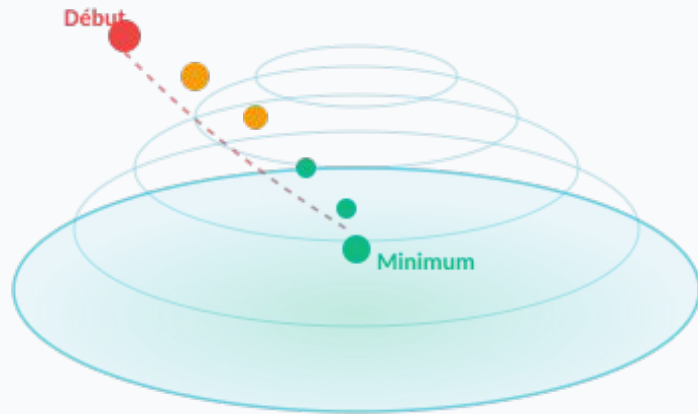
Surface de perte

La balle roule vers le minimum

Le processus itératif

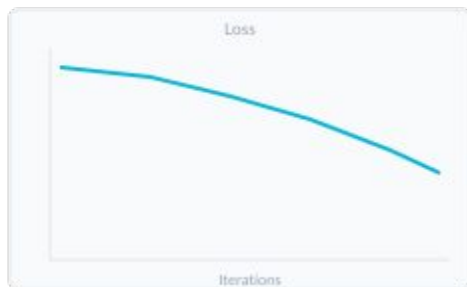
- 1 Calculer la perte avec les poids actuels
- 2 Calculer le gradient (direction de la pente)
- 3 Déplacer les poids dans le sens opposé
- 4 Répéter jusqu'à convergence

$$w_{\text{nouveau}} = w_{\text{ancien}} - \eta \times \nabla L$$



Le Learning Rate (η) — Taille du Pas

Trop petit



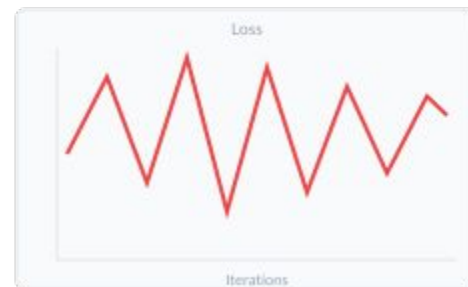
Convergence très lente.
Des milliers d'itérations nécessaires.

Optimal



Convergence rapide et stable.
Le bon compromis.

Trop grand



Oscillations, divergence.
Le modèle ne converge jamais.

Conseil : Commencez avec $\eta = 0.001$ et ajustez. Valeurs typiques : 0.0001, 0.001, 0.01, 0.1



La Règle de la Chaîne (Chain Rule)

Le fondement mathématique qui permet de calculer comment chaque poids du réseau contribue à l'erreur finale, même à travers de multiples couches.

$$\text{Si } z = f(g(x)), \text{ alors } dz/dx = (df/dg) \times (dg/dx)$$

Application dans un réseau de neurones :

$$\partial L / \partial \hat{y}$$

Gradient de la
loss vs sortie

×

$$\partial \hat{y} / \partial z$$

Gradient de
l'activation

×

$$\partial z / \partial w$$

Gradient vs
les poids

$$\partial L / \partial w = (\partial L / \partial \hat{y}) \times (\partial \hat{y} / \partial z) \times (\partial z / \partial w) \rightarrow \text{Comment chaque poids influence la perte}$$



Backpropagation (Rétropropagation)

L'algorithme qui applique la règle de la chaîne couche par couche, de la sortie vers l'entrée, pour calculer le gradient de chaque poids et permettre la mise à jour.

1

Calculer la loss L

Écart entre $\hat{y}$ et y

2

Calculer $\partial L / \partial \hat{y}$

Gradient de la loss

3

Propager en sens inverse

À travers chaque couche

4

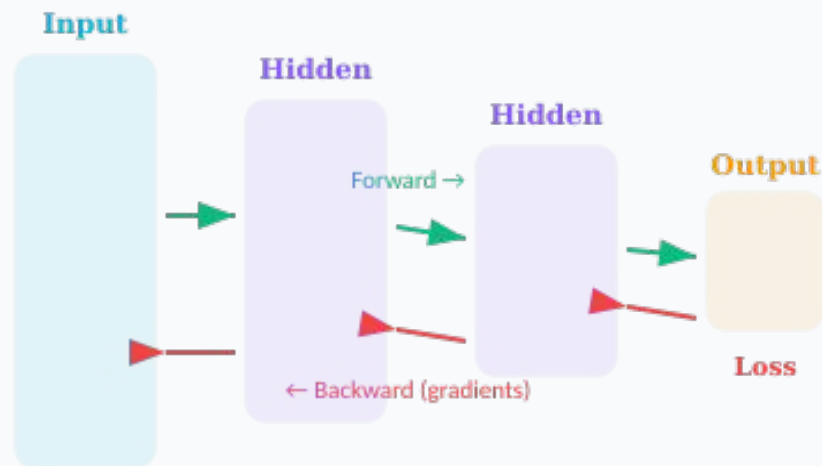
Calculer $\partial L / \partial W$, $\partial L / \partial b$

Pour chaque couche

5

Mettre à jour les poids

$W = W - \eta \times \partial L / \partial W$



TP 3

Boucle d'Entraînement PyTorch

forward

Prédiction



loss

Évaluation



backward

Gradients



step

Mise à jour

```
class MyModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.linear = nn.Linear(2, 3)
    def forward(self, x):
        return self.linear(x)

model = MyModel()
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)

for epoch in range(100):
    optimizer.zero_grad()      # Reset gradients
    outputs = model(inputs)    # Forward pass
    loss = criterion(outputs, labels) # Loss
    loss.backward()            # Backprop
    optimizer.step()           # Update
```




Epochs, Batches et Itérations

Epoch

Un passage complet sur l'ensemble des données d'entraînement. Exemple : 10 epochs = le modèle voit 10 fois chaque image du dataset.

Batch (Mini-lot)

Sous-ensemble de données traité avant chaque mise à jour des poids. Tailles typiques : 32, 64, 128, 256 exemples.

Exemple concret

Dataset : 1 000 images · Batch size : 100 · Entraînement : 50 epochs

→ 1 epoch = 10 batches ($1000 \div 100$)

→ Total = 500 mises à jour des poids (10 batches $\times$ 50 epochs)

Type	Batch Size	Mises à jour / epoch	Caractéristique
Batch complet	N (tout le dataset)	1	Stable mais lent
SGD	1	N	Rapide mais bruité

Le Surapprentissage (Overfitting)

Le modèle mémorise les données d'entraînement au lieu d'en extraire des patterns généralisables. Il excelle sur les données vues mais échoue sur les nouvelles.

Overfitting

Train accuracy : 99%

Test accuracy : 65%

Le modèle a mémorisé chaque détail, y compris le bruit.
Il ne sait pas généraliser.

Bon équilibre

Train accuracy : 92%

Test accuracy : 89%

Le modèle a appris les patterns essentiels sans
mémoriser le bruit. Il généralise bien.

Solutions pour combattre l'overfitting :

Régularisation
L1/L2

Dropout

Data
Augmentation

Early
Stopping

Régularisation L1 et L2

Idée : Ajouter une pénalité à la loss qui décourage les poids trop grands. Plus les poids sont petits, plus le modèle est simple et moins il risque d'overfitter.

L1 (Lasso)

$$\text{Loss_total} = \text{Loss} + \lambda \sum |w_i|$$

- Pousse certains poids vers exactement 0
- Effectue une sélection de features automatique
- Produit un modèle plus simple (sparse)

L2 (Ridge / Weight Decay)

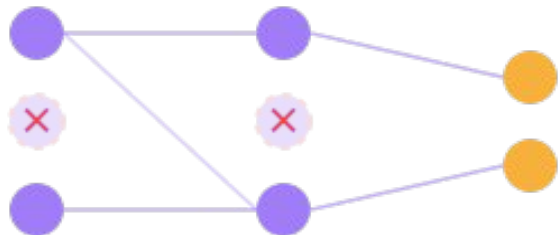
$$\text{Loss_total} = \text{Loss} + \lambda \sum w_i^2$$

- Pousse les poids vers 0 sans les annuler
- Plus courante en Deep Learning
- Implémentée via `weight_decay` dans PyTorch

Dropout et Data Augmentation

Dropout

Pendant l'entraînement, des neurones sont aléatoirement désactivés (mis à 0) à chaque itération. Cela force le réseau à ne pas dépendre de neurones spécifiques.



`nn.Dropout(p=0.5)`

Data Augmentation

Créer artificiellement des variantes des images d'entraînement pour enrichir le dataset et forcer le modèle à être robuste aux transformations.

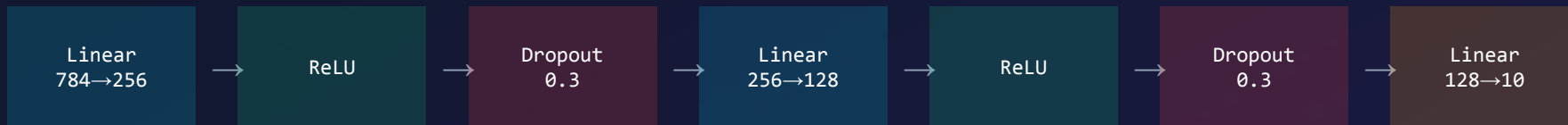
- Rotations aléatoires ($\pm 15^\circ$)
- Retournements horizontaux (flips)
- Zooms et translations
- Changements de luminosité/contraste
- Recadrages aléatoires (random crop)

Le modèle voit plus de variations → meilleure généralisation

TP 4

Amélioration avec Régularisation

Architecture MLP avec Dropout et L2



```
# Optimiseur avec weight_decay (L2)
optimizer = torch.optim.SGD(
    model.parameters(), lr=0.01, weight_decay=0.01
)
```

```
# Entraîner sur MNIST - 10 epochs
for epoch in range(10):
    for images, labels in train_loader:
        images = images.view(-1, 784)
        optimizer.zero_grad()
        output = model(images)
        loss = criterion(output, labels)
        loss.backward()
        optimizer.step()
```



Réseaux Convolutionnels (CNN)

Architecture spécialisée pour les images, inspirée du cortex visuel humain. Les CNN extraient automatiquement une hiérarchie de caractéristiques visuelles.

Structure spatiale

Conserve les relations entre pixels voisins (un MLP perd cette info)

Partage de poids

Un même filtre scanne toute l'image → beaucoup moins de paramètres

Invariance spatiale

Détecte un motif où qu'il soit dans l'image

Hiérarchie

Bords → Textures → Formes → Objets complets

Convolution et Filtres (Kernels)

Un filtre est une petite matrice (ex: 3×3) qui glisse sur l'image. À chaque position, il effectue une multiplication élément par élément puis somme le résultat. Cela produit une feature map.

Image (5×5)		Filtre (3×3)		Feature Map																																											
<table><tr><td>1</td><td>0</td><td>1</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>1</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>1</td><td>0</td><td>1</td></tr></table>	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	×	<table><tr><td>-1</td><td>0</td><td>1</td></tr><tr><td>-1</td><td>0</td><td>1</td></tr><tr><td>-1</td><td>0</td><td>1</td></tr></table>	-1	0	1	-1	0	1	-1	0	1	=	<table><tr><td>0</td><td>2</td><td>0</td></tr><tr><td>-2</td><td>0</td><td>2</td></tr><tr><td>0</td><td>-2</td><td>0</td></tr></table>	0	2	0	-2	0	2	0	-2	0
1	0	1	0	1																																											
0	1	0	1	0																																											
1	0	1	0	1																																											
0	1	0	1	0																																											
1	0	1	0	1																																											
-1	0	1																																													
-1	0	1																																													
-1	0	1																																													
0	2	0																																													
-2	0	2																																													
0	-2	0																																													

1. Positionner le filtre
2. Multiplier élément par élément
3. Sommer → une valeur dans la feature map
4. Déplacer le filtre et répéter

Pooling et Stride/Padding

Max Pooling

Réduit la taille spatiale en gardant la valeur maximale de chaque zone. Rend le réseau plus robuste et rapide.

$\begin{bmatrix} 1 & 3 \\ 4 & 9 \end{bmatrix} \rightarrow \max = 9$

Stride & Padding

Stride : nombre de pixels de déplacement du filtre à chaque pas (1 = pixel par pixel, 2 = on saute).

Padding : ajout de zéros autour de l'image pour conserver sa taille après convolution.

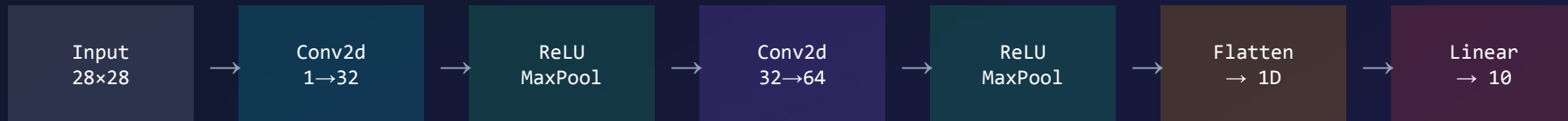
$$\text{Taille de sortie} = (\text{Input} - \text{Filter} + 2 \times \text{Padding}) / \text{Stride} + 1$$

Exemple : Image 28×28, filtre 3×3, padding=1, stride=1 $\rightarrow (28 - 3 + 2 \times 1) / 1 + 1 = 28$ (taille conservée !)

TP 5

Construire votre Premier CNN

Architecture : Classification MNIST (chiffres 0-9)



```
class CNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 32, 3) # 32 filtres 3x3
        self.conv2 = nn.Conv2d(32, 64, 3) # 64 filtres 3x3
        self.pool = nn.MaxPool2d(2) # Réduit /2
        self.fc1 = nn.Linear(64*5*5, 128)
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = x.view(-1, 64*5*5)
        x = F.relu(self.fc1(x))
        return self.fc2(x)
```

Transfer Learning

Réutiliser un modèle déjà entraîné sur un grand dataset (ImageNet, 1.2M images) comme point de départ. On gèle les premières couches et on réentraîne uniquement les dernières pour sa tâche spécifique.



Gain de temps

Pas besoin d'entraîner depuis zéro, des heures → des minutes



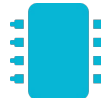
Moins de données

Fonctionne même avec quelques centaines d'images



Meilleure précision

Les features pré-apprises sont universelles et réutilisables



Économie GPU

Moins de calculs, moins de coûts d'infrastructure

Modèles Pré-entraînés Célèbres

ResNet

2015

Skip connections qui résolvent le vanishing gradient. Variantes : 18, 34, 50, 101, 152 couches.

VGG

2014

Architecture très profonde et régulière (que des convolutions 3×3). Variantes : VGG-16, VGG-19.

AlexNet

2012

Premier CNN à dominer ImageNet. 8 couches. A lancé la révolution du Deep Learning.

MobileNet

2017

Optimisé pour mobile et edge. Convolutions séparables en profondeur. Léger et rapide.

```
model = torchvision.models.resnet18(pretrained=True)
```

Introduction à BERT

BERT (Bidirectional Encoder Representations from Transformers) est un modèle de NLP pré-entraîné par Google (2018) qui a révolutionné le traitement du langage naturel.

Caractéristiques

- Architecture Transformer (attention)
- Bidirectionnel : comprend le contexte gauche ET droit simultanément
- Pré-entraîné sur un énorme corpus de texte (Wikipedia + livres)
- Fine-tunable pour des tâches spécifiques avec peu de données

Applications

- Classification de texte et analyse de sentiment
- Question-Answering (Q&A)
- Named Entity Recognition (NER)
- Traduction et résumé automatique

Résumé des Notions Clés

Forward Pass

Propagation des données de l'entrée vers la sortie

Loss Function

MSE (régression) ou Cross-Entropy (classification)

Backpropagation

Calcul des gradients via la règle de la chaîne

Gradient Descent

Mise à jour itérative : $W = W - \eta \times \text{gradient}$

Learning Rate (η)

Vitesse d'apprentissage, typiquement 0.001

Epochs & Batches

Passage complet sur les données / sous-ensemble

Régularisation

L1/L2, Dropout → Combat l'overfitting

CNN

Convolutions + Pooling pour les images

Transfer Learning

Réutiliser des modèles pré-entraînés (ResNet...)

TP 6

Transfer Learning avec ResNet

- Charger ResNet18 pré-entraîné sur ImageNet
- Geler les couches de features (freeze)
- Remplacer la dernière couche pour votre dataset
- Fine-tuner uniquement la dernière couche

```
import torchvision.models as models

# 1. Charger le modèle pré-entraîné
model = models.resnet18(pretrained=True)

# 2. Geler toutes les couches
for param in model.parameters():
    param.requires_grad = False

# 3. Remplacer la dernière couche
model.fc = nn.Linear(512, 10) # 10 classes
```